

Table of Contents

message_types.py	2
Pro_Find.py	4
token_handler.py	7
MessageHandler.py	9
My_WebSocket.py	30

```
from enum import Enum

class MessageTypes(Enum):
    #Anything
        BROAD = "BRD"

    #Authentication
        LOGIN = "LOGIN"
        USER_SIGNUP = "USERSIGNUP"
        FREELANCER_SIGNUP = "FRLNCSIGNUP"
        VERIFICATION = 'VERIFYING'
        FORGOT_PASSWORD_REQUEST = "FRGT_PAS_RQT"
        FORGOT_PASSWORD_AUTHENTICATION = "FRGT_PASS_AUTH"
        CHANGE_PASS = "CHNG_PAS"

    #Homepage
        GET_USER_INFO = "USR_INFO"
        MARK_READ_NOTIFICATION = "RD_NTFC"
        UPDATE_APPOINTMENTS_STATUS= "UPDT_APP_STAT"
        GET_APPOINTMENT_TIMES = "GET_APP_TIME"
        MAKE_APPOINTMENT="MK_APP"

    #Search
        GET_PUBLIC_PROFILE_INFO= "PBL_INFO"
        GET_MINIMAL_FREELANCER_INFO="MNML_VIEW"
        GET_JOBS="GET_JBS"
        GET_CITIES_BY_JOB="GET_CITIS"

class StatusMessage(Enum):
    #Anything
        TOKEN_BAD = "BAD_TKN"

    #Authentication
        LOGGED_IN = "LOGGED"
        FAILED_LOG_IN = "FAILED_LOG"

        SIGNING_UP = "SIGNED"
        FAILED_SIGN_UP = "FAILED_SIGNED"

        VERIFICATION_GOOD = "VERI_GOOD"
        VERIFICATION_BAD = "VERI_BAD"

        FORGOT_PASSWORD_GOOD = "FRGT_GOOD"
        FORGOT_PASSWORD_BAD = "FRGT_BAD"

        CHANGE_PASSWORD_GOOD = "CHNG_GOOD"
        CHANGE_PASSWORD_BAD = "CHNG_BAD"

    #Homepage
        GOT_USER_INFO = "GOT_USR_INFO"
        FAILED_TO_GET_USER_INFO = "FLD_USR_INFO"

        MARKED_READ_NOTIFICATIONS = "READ_NOTIFICATIONS"
        FAILED_TO_MARK_READ_NOTIFICATIONS = "FAILED_READ_NOTIFICATIONS"

        UPDATED_APP_STATUS="UPDT_APP"
        FAILED_TO_UPDATE_APP_STATUS="FAILED_UPDT_APP"

        GOT_APPOINTMENT_TIMES="GOT_APP_TIME"
        FAILED_TO_GET_APPOINTMENT_TIMES = "FLD_APP_TIME"
```

```
BOOKED_APPOINTMENT="BKD_APP"  
FAILED_TO_BOOK_APPOINTMENT="FLD_BKD_APP"
```

```
#Search
```

```
GOT_JOBS="GOT_JBS"  
FAILED_TO_GET_JOBS="FLD_JBS"
```

```
GOT_CITIES="GOT_CITIS"  
FAILED_TO_GET_CITIES="FLD_CITIS"
```

```
GOT_MINIMAL_INFO= "GOT_MNML"  
FAILED_TO_GET_MINIMAL_INFO="FLD_MNML"
```

```
GOT_FREELANCER_PUBLIC_INFO= "GOT_FREE"  
FAILED_TO_GET_FREELANCER_PUBLIC_INFO="FLD_FREE"
```

```

import socket
import threading
import My_WebSocket
import MessageHandler
from datetime import datetime, timedelta
from collections import deque
import time
import sqlite3

MAX_MESSAGES = 15
TIME_WINDOW = 20

dispatcher = MessageHandler.configure_dispatcher()
DATABASE = r"C:\Coding\pro-find\Python\my_app.db"

MAX_CONNECTIONS_PER_IP = 5
IP_TIMEOUT = 60

ip_connections = {}

def can_accept_connection(ip: str):
    now = time.time()
    if ip in ip_connections:
        ip_connections[ip] = [t for t in ip_connections[ip] if now - t < IP_TIMEOUT]

        if len(ip_connections[ip]) >= MAX_CONNECTIONS_PER_IP:
            return False

        ip_connections[ip].append(now)
        return True
    else:
        ip_connections[ip] = [now]
        return True

def cleanup_tables():
    while True:
        try:
            now = datetime.now()
            current_date = now.strftime("%Y-%m-%d")
            current_time = now.strftime("%H:%M")
            one_week_ago = (now - timedelta(days=7)).strftime("%Y-%m-%d %H:%M:%S")
            two_day_ago = (now - timedelta(days=2)).strftime("%Y-%m-%d %H:%M:%S")

            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()

                cur.execute("""
                    DELETE FROM pending_users
                    WHERE expires_at IS NOT NULL AND expires_at <= CURRENT_TIMESTAMP
                """)

                cur.execute("""DELETE FROM appointments WHERE date < ? OR (date = ? AND start_
time < ?)
                """, (current_date, current_date, current_time))

```

```

        cur.execute("""DELETE FROM notifications WHERE created_at < ? OR (created_at <
? AND is_read = 1)
        """, (one_week_ago, two_day_ago))

        conn.commit()

    except Exception as e:
        print(f"Cleanup error - {e}")

    time.sleep(3600)

def handle_client(soc: socket.socket):
    clt_soc = My_WebSocket.accept_client(soc)

    message_times = deque()

    while True:
        try:
            while message_times and time.time() - message_times[0] > TIME_WINDOW:
                message_times.popleft()

            if len(message_times) >= MAX_MESSAGES:
                print("Rate limit exceeded, ignoring message")
                time.sleep(0.5)
                continue

            payload = My_WebSocket.recv_message(clt_soc)

            message_times.append(time.time())

            msg = MessageHandler.Message(payload)
        except Exception as e:
            print(f"Exception in recieving - {e}")
            break

        print(f"Got - {msg.type_of}, {msg.data}\n\n")
        handler_type, to_send = dispatcher.dispatch(msg)
        print(f"Sending - {handler_type}, {to_send}\n\n")
        My_WebSocket.send_message(
            clt_soc,
            handler_type,
            msg.token,
            to_send,
            False
        )

def main_thread(srv_soc: socket.socket):
    while True:
        clt, addr = srv_soc.accept()
        ip = addr[0]
        if not can_accept_connection(ip):
            print(f"Too many connections from {ip}, rejecting")
            clt.close()
            continue
        t = threading.Thread(target=handle_client, args=(clt,), daemon=True)
        t.start()

```

```
if __name__ == "__main__":  
    srv = socket.socket()  
    srv.bind(("0.0.0.0", 1111))  
    srv.listen(10)  
  
    cleanup_thread = threading.Thread(target=cleanup_tables, daemon=True)  
    cleanup_thread.start()  
  
    main_thread(srv)
```

```
from json import loads, dumps
from hashlib import sha256
from base64 import b64decode, b64encode
import sqlite3
from datetime import datetime, timedelta

sec = b"CYBERISH"

DATABASE = r"C:\Coding\pro-find\Python\my_app.db"

def decode(token: str):
    token_split = token.split(".")
    if len(token_split)!=2:
        return None

    enc_data,enc_sign = token_split
    try:
        dec_data = b64decode(enc_data)
        data = loads(dec_data)
        provided_sign = b64decode(enc_sign)
    except:
        return None

    calc_sign = sha256(sec + dec_data).hexdigest().encode()
    if provided_sign != calc_sign:
        return None
    return data

def is_token_valid(token:str):
    dec_tkn = decode(token)

    if not dec_tkn:
        return False
    exp = datetime.fromtimestamp(dec_tkn["exp"])

    if exp < datetime.now():
        return False
    return True

def _encode(data):
    dec_data = dumps(data).encode()
    enc_data = b64encode(dec_data).decode()
    dec_sign = sha256(sec + dec_data).hexdigest().encode()
    enc_sign = b64encode(dec_sign).decode()
    return f"{enc_data}.{enc_sign}"

def create_token(user_email:str):
    with sqlite3.connect(DATABASE) as conn:
        cur = conn.cursor()

        cur.execute("SELECT id, full_name, created_at FROM users WHERE email=?", (user_email,))
    )
    row = cur.fetchone()
    if not row:
        return None
```

```
        id, name, created_at = row
        tkn_obj = {"id":id, "email": user_email, "name":name, "Creation time": created_at, "exp":int((datetime.now()+timedelta(days=1)).timestamp())}
        return _encode(tkn_obj)
```



```
import sqlite3
import hashlib
import os
from message_types import MessageTypes, StatusMessage
from abc import ABC, abstractmethod
import re
from datetime import datetime, timedelta
import random
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from token_handler import is_token_valid, create_token, decode
from collections import defaultdict
from os import getenv
from dotenv import load_dotenv
import bleach

#region Consts
DATABASE = r"C:\Coding\pro-find\Python\my_app.db"
load_dotenv()

EMAIL = getenv("EMAIL")
PEPPER = getenv("PEPPER")
AUTHENTICATION_PAS = getenv("AUTHENTICATION_PAS")

with open(r"C:\Coding\pro-find\Python\professional.txt", 'r') as f:
    PROFESSIONS = [line.strip() for line in f]
with open(r"C:\Coding\pro-find\Python\cities.txt", 'r') as f:
    LOCALITIES = [line.strip() for line in f]

#endregion

def hash_password(password:str, salt:str) ->str:

    combined_encoded= (salt+password+PEPPER).encode()
    return hashlib.sha256(combined_encoded).hexdigest()

class Message:
    def __init__(self, payload):
        self.type_of = payload["type"]
        self.data = payload["data"]
        self.token = payload["token"]

class MessageHandler(ABC):
    @abstractmethod
    def handle(self, msg:Message) ->tuple:
        pass

#region Dispatcher
class MessageDispatcher:
    no_token_check = {
        MessageTypes.LOGIN.value, MessageTypes.USER_SIGNUP.value,
```

```

        MessageTypes.FREELANCER_SIGNUP.value, MessageTypes.VERIFICATION.value,
        MessageTypes.FORGOT_PASSWORD_AUTHENTICATION.value, MessageTypes.FORGOT_PAS
SWORD_REQUEST.value,
        MessageTypes.CHANGE_PASS.value}

    def __init__(self):
        self._handlers={}

    def register(self, msg_type, cls):
        self._handlers[msg_type] = cls

    def dispatch(self, msg:Message) ->tuple:
        if msg.type_of not in self.no_token_check:
            if not is_token_valid(msg.token):
                return MessageTypes.BROAD, {StatusMessage.TOKEN_BAD.value:"Token format invali
d"}

        handler_class = self._handlers.get(msg.type_of)

        if not handler_class:
            raise Exception("No handler registered")

        handler:MessageHandler = handler_class()
        return handler.handle(msg)

def configure_dispatcher() -> MessageDispatcher:
    d = MessageDispatcher()
    d.register(MessageTypes.LOGIN.value, LoginDispatcher)
    d.register(MessageTypes.USER_SIGNUP.value, UserSignUpService)
    d.register(MessageTypes.FREELANCER_SIGNUP.value, FreelancerSignUpService)
    d.register(MessageTypes.VERIFICATION.value, VerificationDispatcher)
    d.register(MessageTypes.FORGOT_PASSWORD_REQUEST.value, ForgotPasswordEmailVerifciationDisp
atcher)
    d.register(MessageTypes.FORGOT_PASSWORD_AUTHENTICATION.value, ForgotPasswordAuthentication
Dispatcher)
    d.register(MessageTypes.CHANGE_PASS.value, UpdatePasswordDispatcher )
    d.register(MessageTypes.GET_USER_INFO.value, UserInfoDispatcher)
    d.register(MessageTypes.UPDATE_APPOINTMENTS_STATUS.value, ChangeAppointmentStatusDispatche
r)
    d.register(MessageTypes.MARK_READ_NOTIFICATION.value, MarkNotificationsReadDispatcher)
    d.register(MessageTypes.GET_APPOINTMENT_TIMES.value, AppointmentStartTimesDispatcher)
    d.register(MessageTypes.MAKE_APPOINTMENT.value, BookAppointmentDispatcher)
    d.register(MessageTypes.GET_JOBS.value, ExistingJobsDispatcher)
    d.register(MessageTypes.GET_CITIES_BY_JOB.value, CitiesByJobDispatcher)
    d.register(MessageTypes.GET_MINIMAL_FREELANCER_INFO.value, MinimalProfessionalInfo)
    d.register(MessageTypes.GET_PUBLIC_PROFILE_INFO.value, FreelancerPublicInfo)
    return d

#region authentication
class LoginDispatcher(MessageHandler):
    def handle(self, msg:Message) -> tuple:
        try:

            email = msg.data["email"]
            password = msg.data["password"]
            with sqlite3.connect(DATABASE) as conn:

```

```

        cursor = conn.cursor()

        cursor.execute("SELECT password_hash, salt FROM users WHERE email = ?", (email
,))

        data = cursor.fetchone()

        if data:
            db_password, salt = data
            hashed_ps = hash_password(password, salt)
            if db_password == hashed_ps:
                return MessageTypes.LOGIN, {StatusMessage.LOGGED_IN.value:create_token
(email)}

            return MessageTypes.LOGIN, {StatusMessage.FAILED_LOG_IN.value:"Email or passwo
rd incorrect"}

        except Exception as e:
            print("Login dispatcher - ", e)
            return MessageTypes.LOGIN, {StatusMessage.FAILED_LOG_IN.value:"Client error"}


class UserSignUpService(MessageHandler):
    def handle(self, msg:Message) -> tuple:
        v = UserSignUpValidator()
        info_is_good, message = v.validate(msg)
        if not info_is_good:
            return MessageTypes.USER_SIGNUP, message

        status, message = self._add_pending_user(msg)
        if not status:
            return MessageTypes.USER_SIGNUP, message

        return MessageTypes.USER_SIGNUP, {StatusMessage.SIGNING_UP.value:""}

    def _add_pending_user(self, msg:Message) -> tuple[bool,dict]:
        try:
            data = msg.data
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                conn.execute("BEGIN")

                cur.execute("SELECT 1 FROM users WHERE email=?", (data["email"],))

                if cur.fetchone():
                    conn.rollback()
                    return False, {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}

                cur.execute("""SELECT expires_at FROM pending_users WHERE email=?""", (data["e
mail"],))

                find = cur.fetchone()

                if find:
                    expires_at = datetime.strptime(find[0], "%Y-%m-%d %H:%M:%S.%f")

                    if expires_at<datetime.now():
                        cur.execute("""DELETE FROM pending_users WHERE email=?""", (data["emai
l"],))

                    else:
                        conn.rollback()

```

```

        return False, {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}
    except Exception as e:
        conn.rollback()
        print("User sign up service - ", e)
        return False, {StatusMessage.FAILED_SIGN_UP.value: "Not all fields were valid"}

class FreelancerSignUpService(MessageHandler):
    def handle(self, msg:Message) -> tuple:
        v = FreelancerSignUpValidator()
        info_is_good, message = v.validate(msg)
        if not info_is_good:
            return MessageTypes.FREELANCER_SIGNUP, message

        status, message = self._add_pending_freelancer(msg)
        if not status:
            return MessageTypes.FREELANCER_SIGNUP, message

        return MessageTypes.FREELANCER_SIGNUP, {StatusMessage.SIGNING_UP.value:""}

    def _add_pending_freelancer(self, msg:Message) -> tuple[bool,dict]:
        data = msg.data
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                conn.execute("BEGIN")

                cur.execute("SELECT 1 FROM users WHERE email=?", (data["email"],))

                if cur.fetchone():
                    conn.rollback()
                    return False, {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}

                cur.execute("SELECT expires_at FROM pending_users WHERE email=?", (data["email"]

```

```

    ],))

    row = cur.fetchone()

    if row:
        expires_at = datetime.strptime(row[0], "%Y-%m-%d %H:%M:%S.%f")
        if expires_at < datetime.now():
            cur.execute("""DELETE FROM pending_users WHERE email=?""", (data["email"],))
        else:
            conn.rollback()
            return False, {StatusMessage.FAILED_SIGN_UP.value: "Email already in use"}

    salt = os.urandom(16).hex()
    password_hash = hash_password(data["password"], salt)
    ver = EmailVerification(data["email"])
    cities = ", ".join(data["cities"])

    cur.execute("""INSERT INTO pending_users (full_name, email, password_hash,
        salt, user_type, profession, cities, years, job_duration, description,
        start_working, finish_working, verification_code, expires_at, hour_price) VALUES (
        ?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
        (data["name"], data["email"], password_hash, salt, data["role"], data["profession"],
        cities, data["years"], data["jobDuration"], data["description"], data["startWorking"],
        data["finishWorking"], ver.verification_code, ver.expires_at, data["hourPrice"])))

    if not ver.send_verification_code():
        conn.rollback()
        return False, {StatusMessage.FAILED_SIGN_UP.value: "Failed to send verification email. Check email field"}

    conn.commit()
    return True, {MessageTypes.VERIFICATION.value: "Sending verification"}

except sqlite3.IntegrityError:
    return False, {StatusMessage.FAILED_SIGN_UP.value: "Email already in use"}

except Exception as e:
    conn.rollback()
    print("Freelancer sign up service -", e)
    return False, {StatusMessage.FAILED_SIGN_UP.value: "Not all fields were valid"}

class VerificationDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        verified, message = EmailVerification.check_verification_code(msg)
        if verified:
            if self.log_new_customer(msg):
                return MessageTypes.VERIFICATION, message
            else:
                return MessageTypes.VERIFICATION, {StatusMessage.VERIFICATION_BAD.value: "Error while logging user"}
        else:
            return MessageTypes.VERIFICATION, message

```

```

def log_new_customer(self, msg:Message) -> bool:
    try:
        with sqlite3.connect(DATABASE) as conn:
            conn.execute("BEGIN")
            cur = conn.cursor()

            data = msg.data
            email = data["email"]
            role = data["role"]

            if role == "User":
                cur.execute("SELECT full_name, password_hash, salt, user_type FROM pending_users WHERE email=?", (email,))
                row = cur.fetchone()
                if not row:
                    conn.rollback()
                    return False

                name, password_hash, salt, user_type = row

                cur.execute("""INSERT INTO users (full_name, email, password_hash, user_type, salt)
                                VALUES (?, ?, ?, ?, ?)""", (name, email, password_hash, user_type, salt))

            elif role == "Freelancer":
                cur.execute("""SELECT full_name, password_hash, salt, user_type, profession, cities, years, job_duration, description, start_working, finish_working, hour_price FROM pending_users WHERE email=?""", (email,))
                row = cur.fetchone()
                if not row:
                    conn.rollback()
                    return False

                (name, password_hash, salt, user_type, profession, cities, years, job_duration, description, start_working, finish_working, hour_price) = row

                cur.execute("""INSERT INTO users (full_name, email, password_hash, user_type, salt)
                                VALUES (?, ?, ?, ?, ?)""", (name, email, password_hash, user_type, salt))

                id = cur.lastrowid

                cur.execute("""INSERT INTO professional (user_id, profession, service_cities, description, years_experience, avg_job_duration, start_time, end_time, hour_price)
                                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""", (id, profession, cities, description, years, job_duration, start_working, finish_working, hour_price))

                cur.execute("DELETE FROM pending_users WHERE email=?", (email,))
                conn.commit()
                return True

    except sqlite3.IntegrityError:
        conn.rollback()
        return False

    except Exception as e:
        conn.rollback()

```

```

        print("Verification dispatcher, logging customer -", e)
        return False

class ForgotPasswordEmailVerficiationDispatcher(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                conn.execute("BEGIN")
                email = msg.data["email"]
                cur = conn.cursor()

                cur.execute("SELECT 1 FROM users WHERE email=?", (email,))
                if not cur.fetchone():
                    return MessageTypes.FORGOT_PASSWORD_REQUEST, {StatusMessage.FORGOT_PASSWORD_BAD.value:"System error"}

                cur.execute("SELECT expires_at FROM pending_users WHERE email=?", (email,))

                find =cur.fetchone()
                if find:

                    expires_at = datetime.strptime(find[0], "%Y-%m-%d %H:%M:%S.%f")
                    if expires_at<datetime.now():
                        cur.execute("DELETE FROM pending_users WHERE email=?", (email,))
                    else:
                        return MessageTypes.FORGOT_PASSWORD_REQUEST, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Expired verification time"}

                ver = EmailVerification(email)

                if not ver.send_verification_code():
                    conn.rollback()
                    return MessageTypes.FORGOT_PASSWORD_REQUEST, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Error sending verification email\ntry again at a later time"}

                cur.execute("""
                    INSERT INTO pending_users
                    (full_name, email, password_hash, salt, user_type, verification_code, expires_at)

                    VALUES (?, ?, ?, ?, ?, ?, ?)
                    """, ("", email, "", "", "User", ver.verification_code, ver.expires_at))

                conn.commit()
                return MessageTypes.FORGOT_PASSWORD_REQUEST, {StatusMessage.FORGOT_PASSWORD_GOOD.value:""}

        except Exception as e:
            conn.rollback()
            print("Forgot Password Dispatcher - ",e)
            return MessageTypes.FORGOT_PASSWORD_REQUEST, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Client error"}

```

```

class ForgotPasswordAuthenticationDispatcher(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                conn.execute("BEGIN")
                cur = conn.cursor()
                email = msg.data["email"]
                ver_code = msg.data["veri_code"]

                cur.execute("SELECT verification_code, expires_at FROM pending_users WHERE email=?", (email,))
                pending = cur.fetchone()

                if not pending:
                    return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Email is not verifying"}

                cur.execute("SELECT 1 FROM users WHERE email=?", (email,))
                user = cur.fetchone()
                if not user:
                    return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_BAD.value:"No such user in system"}

                expires_at = datetime.strptime(pending[1], "%Y-%m-%d %H:%M:%S.%f")
                if expires_at<datetime.now():
                    cur.execute("DELETE FROM pending_users WHERE email=?", (email,))
                    conn.commit()
                    return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Verification time finished"}

                if ver_code == pending[0]:
                    cur.execute("DELETE FROM pending_users WHERE email=?", (email,))
                    cur.execute("INSERT INTO pass_change (email) VALUES (?)", (email,))
                    conn.commit()
                    return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_GOOD.value:""}
                else:
                    return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Incorrect verification code"}

        except Exception as e:
            conn.rollback()
            print("Forgot password authentication dispatcher - ", e)
            return MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, {StatusMessage.FORGOT_PASSWORD_BAD.value:"Client error"}

class UpdatePasswordDispatcher(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                conn.execute("BEGIN")
                cur = conn.cursor()
                email = msg.data["email"]
                password = msg.data["password"]

```



```

        cur.execute("""SELECT 1 FROM pass_change WHERE email=?""", (email,))
        user = cur.fetchone()

        if not user:
            return MessageTypes.CHANGE_PASS, {StatusMessage.CHANGE_PASSWORD_BAD.value:
"User not changin password"}

        salt = os.urandom(16).hex()
        password_hash = hash_password(password, salt)

        cur.execute(""" UPDATE users SET password_hash = ?, salt = ? WHERE email = ?""
", (password_hash, salt, email))
        cur.execute("""DELETE FROM pass_change WHERE email=?""", (email,))
        conn.commit()
        return MessageTypes.CHANGE_PASS, {StatusMessage.CHANGE_PASSWORD_GOOD.value:""}

    except Exception as e:
        conn.rollback()
        print("Update password dispatcher - ", e)
        return MessageTypes.CHANGE_PASS, {StatusMessage.CHANGE_PASSWORD_BAD.value:"Client
error"}
#endregion

#region homepage
class UserInfoDispatcher(MessageHandler):
    def handle(self, msg:Message)->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                actual_email = decode(msg.token)["email"]
                cur = conn.cursor()
                email = msg.data["email"]

                if email !=actual_email:
                    return MessageTypes.GET_USER_INFO, {StatusMessage.FAILED_TO_GET_USER_INFO.
value:"Email not matching token"}

                cur.execute("SELECT id, full_name, user_type FROM users WHERE email=?", (email
,))
                row = cur.fetchone()

                if not row:
                    return MessageTypes.GET_USER_INFO, {StatusMessage.FAILED_TO_GET_USER_INFO.
value:"Couldn't find user"}

                id, name, role = row
                payload_to_send = {"name":name, "role":role}

                if role=="Freelancer":
                    cur.execute("SELECT profession, service_cities, description, years_experie
nce, rating, start_time, end_time, avg_job_duration, hour_price FROM professional WHERE user_id
=?",(id,))
                    row = cur.fetchone()
                    if not row:
                        return MessageTypes.GET_USER_INFO, {StatusMessage.FAILED_TO_GET_USER_I
NFO.value:"Couldn't find user"}
                    profession, cities, description, years_experience, rating, start, end, job
_duration, hour_price = row

```

```

        try:
            h, m = map(int, job_duration.split(':'))

            h_str = f"{h}h" if h > 0 else ""
            m_str = f"{m}m" if m > 0 else ""

            job_duration = f"{h_str} {m_str}".strip()
        except:
            pass

        payload_to_send.update({"job":profession, "cities":cities, "description":description, "years":years_experience, "rating":rating, "start_working":start, "end_working":end, "job_duration":job_duration, "hour_price":hour_price})

        appointments = self._get_appointments(id, cur, role)

        notifications = self._get_notifications(id, cur)

        conn.commit()
        payload_to_send["appointments"] = appointments
        payload_to_send["notifications"] = notifications

        return MessageTypes.GET_USER_INFO, {StatusMessage.GOT_USER_INFO.value:payload_to_send, "user_id": id}

    except Exception as e:
        print(f"User info dispatcher - {e}")
        return MessageTypes.GET_USER_INFO, {StatusMessage.FAILED_TO_GET_USER_INFO.value:"Couldn't find user"}

def _get_appointments(self, user_id, cur, role) -> list:
    is_pro = True if role == "Freelancer" else False

    now = datetime.now()
    today = now.strftime("%Y-%m-%d")
    time_now = now.strftime("%H:%M")

    if is_pro:
        query = """
            SELECT u.full_name, a.date, a.id, a.start_time, a.end_time,
                   a.address, a.details, a.status
            FROM appointments a
            JOIN users u ON a.customer_id = u.id
            WHERE a.professional_id = ?
        """
    else:
        query = """
            SELECT u.full_name, a.date, a.id, a.start_time, a.end_time,
                   a.address, a.details, a.status
            FROM appointments a
            JOIN professional p ON a.professional_id = p.user_id
            JOIN users u ON p.user_id = u.id
            WHERE a.customer_id = ? AND a.status = 'Confirmed'
        """

    query += """
        AND (a.date > ? OR (a.date = ? AND a.start_time >= ?))
        ORDER BY a.date ASC, a.start_time ASC
    """

```

```

        cur.execute(query, (user_id, today, today, time_now))
        rows = cur.fetchall()

        results = []
        for r in rows:
            d_parts = str(r[1]).split('-')
            display_date = f"{d_parts[2]}/{d_parts[1]}/{d_parts[0]}" if len(d_parts) == 3
else r[1]

            results.append({
                "id": r[2],
                "person_name": r[0],
                "display_date": display_date,
                "date": str(r[1]),
                "start_time": str(r[3]),
                "end_time": str(r[4]),
                "address": r[5],
                "details": r[6],
                "status": r[7],
                "timestamp": f"{r[1]} {r[3]}"
            })

        return results

def _get_notifications(self, user_id, cur) -> list:
    query = """
        SELECT n.message, n.created_at, n.is_read, u.full_name as sender_name
        FROM notifications n
        LEFT JOIN users u ON n.from_id = u.id
        WHERE n.user_id = ?
        ORDER BY n.created_at DESC
    """
    cur.execute(query, (user_id,))
    rows = cur.fetchall()

    notifications = []
    for r in rows:
        message, raw_date, is_read, sender_name = r
        try:
            dt_obj = datetime.strptime(raw_date, "%Y-%m-%d %H:%M:%S.%f")
            formatted_date = dt_obj.strftime("%d/%m/%y")
        except ValueError:
            try:
                dt_obj = datetime.strptime(raw_date, "%Y-%m-%d %H:%M:%S")
                formatted_date = dt_obj.strftime("%d/%m/%y")
            except Exception:
                formatted_date = str(raw_date)[:10]

        notifications.append({
            "message": message,
            "created_at": formatted_date,
            "from_name": sender_name if sender_name else "System",
            "is_read": bool(is_read)
        })

    return notifications

class MarkNotificationsReadDispatcher(MessageHandler):

```

```

def handle(self, msg: Message) -> tuple:
    try:
        user_id = msg.data["user_id"]
        with sqlite3.connect(DATABASE) as conn:
            cur = conn.cursor()
            cur.execute("UPDATE notifications SET is_read = 1 WHERE user_id = ? AND is_read
= 0", (user_id,))
            conn.commit()
        return MessageTypes.MARK_READ_NOTIFICATION, {StatusMessage.MARKED_READ_NOTIFICATION
NS.value: ""}
    except Exception as e:
        print("Marked notification dispatcher error - ", e)
        return MessageTypes.MARK_READ_NOTIFICATION, {StatusMessage.FAILED_TO_MARK_READ_NOT
IFICATIONS.value:"Failed to mark messages"}

```

```

class ChangeAppointmentStatusDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                apps = msg.data["appointments"]

                worked = []
                failed = []

                first_key = int(list(apps.keys())[0])
                cur.execute("SELECT u.full_name, a.professional_id FROM appointments a JOIN use
rs u ON a.professional_id = u.id WHERE a.id = ?", (first_key,))
                res = cur.fetchone()
                if not res:
                    prof_name = "Freelancer"
                    p_id = None
                else:
                    prof_name, p_id = res

                status_map = {'accepted': "Confirmed", 'cancelled': "Cancelled"}

                for app_id, status_input in apps.items():
                    try:
                        target_status = status_map.get(status_input)

                        if target_status == "Confirmed":
                            if self._is_time_slot_taken(cur, app_id):
                                cur.execute("DELETE FROM appointments WHERE id=?", (app_id,))
                                failed[app_id] = "Slot already taken"
                                continue

                        cur.execute("SELECT customer_id FROM appointments WHERE id=?", (app_id
,))

                        row = cur.fetchone()
                        if not row:
                            failed[app_id] = "Appointment not found"
                            continue

                        u_id = row[0]

                        if target_status == "Confirmed":
                            cur.execute("UPDATE appointments SET status=? WHERE id=?", (target
_status, app_id))

                            msg_text = f"{prof_name} has accepted your appointment"

```

```

        else:
            cur.execute("DELETE FROM appointments WHERE id=?", (app_id,))
            msg_text = f"{prof_name} has declined your appointment"

            cur.execute("""INSERT INTO notifications (user_id, message, is_read, c
reated_at, from_id)
                        VALUES (?, ?, 0, ?, ?)""",
                        (u_id, msg_text, datetime.now().strftime("%Y-%m-%d %H:%M:%
S"), p_id))

            worked.append(app_id)

        except Exception as inner_e:
            failed[app_id] = str(inner_e)
            continue

    conn.commit()

    return MessageTypes.UPDATE_APPOINTMENTS_STATUS, {
        StatusMessage.UPDATED_APP_STATUS.value: "Batch processed",
        "results": {"processed":worked, "Failed":failed}
    }

except Exception as e:
    print(f"Appointment Status change Dispatcher Error: {e}")
    return MessageTypes.UPDATE_APPOINTMENTS_STATUS, {StatusMessage.FAILED_TO_UPDATE_AP
P_STATUS.value: "System error"}

def _is_time_slot_taken(self, cur, app_id):
    cur.execute("SELECT professional_id, date, start_time, end_time FROM appointments WHERE
id=?", (app_id,))
    target = cur.fetchone()
    if not target:
        return False

    p_id, a_date, a_start, a_end = target

    query = """SELECT id FROM appointments WHERE professional_id = ? AND date = ?
AND status = 'Confirmed' AND id != ? AND (? < end_time AND start_time < ?)
"""
    cur.execute(query, (p_id, a_date, app_id, a_start, a_end))
    return cur.fetchone() is not None

class AppointmentStartTimesDispatcher(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                pro_id = msg.data["id"]

                cur.execute("SELECT avg_job_duration, start_time, end_time FROM professional WH
ERE user_id=?", (pro_id,))
                row = cur.fetchone()
                if not row:
                    return MessageTypes.GET_APPOINTMENT_TIMES, {StatusMessage.FAILED_TO_GET_AP
POINTMENT_TIMES.value:"Couldn't find freelancer"}

                job_duration, start_time, end_time = row

                cur.execute("""SELECT start_time, date FROM appointments WHERE professional_id

```

=?

```

        AND date BETWEEN date('now') AND date('now', '+1 month')
        AND status='Confirmed' ORDER BY date ASC; """ , (pro_id,))

    existing_appointments = {}
    for row in cur.fetchall():
        time, date = row[0], row[1]
        if date not in existing_appointments:
            existing_appointments[date] = []
            existing_appointments[date].append(time)

    existing_appointments = dict(existing_appointments)

    h,m = map(int, job_duration.split(":"))
    duration_minutes = (h*60) + m

    app_times = {}
    current_date = datetime.now() + timedelta(days=1)

    for _ in range(0, 35):
        date_str = current_date.strftime("%Y-%m-%d")
        booked_slots = existing_appointments.get(date_str, [])
        app_times[date_str] = _calculate_free_day_working_hours(start_time, end_time, duration_minutes, booked_slots)
        current_date = current_date+timedelta(days=1)

    return MessageTypes.GET_APPOINTMENT_TIMES, {StatusMessage.GOT_APPOINTMENT_TIMES.value:app_times}
except Exception as e:
    print("Appointment time dispatcher error - ", e)
    return MessageTypes.GET_APPOINTMENT_TIMES, {StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES.value:"Issue fetching the appointment dates and times"}

def _calculate_free_day_working_hours(start_time, end_time, job_duration, existing_appointments=None) ->list:
    if existing_appointments is None:
        existing_appointments=[]

    format = "%H:%M"
    current_slot = datetime.strptime(start_time, format)
    end_of_day = datetime.strptime(end_time, format)

    available_times = []
    while current_slot+timedelta(minutes=job_duration) <= end_of_day:
        slot_str = current_slot.strftime(format)

        if slot_str not in existing_appointments:
            available_times.append(slot_str)

        current_slot+= timedelta(minutes=job_duration)

    return available_times

class BookAppointmentDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:

```

```

        cur = conn.cursor()
        app = msg.data["app"]

        cur.execute("SELECT avg_job_duration FROM professional WHERE user_id = ?", (app["prof_id"],))
        row = cur.fetchone()
        if not row:
            return MessageTypes.MAKE_APPOINTMENT, {StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Freelancer not found"}

        duration_str = row[0]
        h, m = map(int, duration_str.split(':'))
        duration_minutes = (h * 60) + m

        end_time_str = self.calculate_end_time(app["start_time"], duration_minutes)

        cur.execute("SELECT start_time FROM appointments WHERE date = ? AND professional_id = ? AND status = 'Confirmed'", (app["date"], app["prof_id"]))
        start_times_booked = [row[0] for row in cur.fetchall()]

        available_slots = _calculate_free_day_working_hours(app["start_time"], end_time_str, duration_minutes, start_times_booked)

        if app["start_time"] not in available_slots:
            return MessageTypes.MAKE_APPOINTMENT, {StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Time slot is no longer available."}

        clean_details = bleach.clean(app["details"], tags=[], attributes={}, strip=True)
        clean_address = bleach.clean(app["address"], tags=[], attributes={}, strip=True)

        cur.execute("""
            INSERT INTO appointments (professional_id, customer_id, date, start_time, end_time, address, details, status)
            VALUES (?, ?, ?, ?, ?, ?, ?, 'Requested')""", (app["prof_id"], app["user_id"], app["date"], app["start_time"], end_time_str, clean_address, clean_details))

        cur.execute("""INSERT INTO notifications (user_id, message, is_read, created_at)
            VALUES (?, ?, 0, ?)""", (app["prof_id"], "You have a pending appointment waiting for you", datetime.now().strftime("%Y-%m-%d %H:%M:%S")))

        conn.commit()
        return MessageTypes.MAKE_APPOINTMENT, {StatusMessage.BOOKED_APPOINTMENT.value: "Appointment requested successfully"}

    except Exception as e:
        print(f"CRITICAL: Appointment booking failure - {e}")
        return MessageTypes.MAKE_APPOINTMENT, {StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Server error, please try again."}

    def calculate_end_time(self, start_time_str, duration_minutes):
        fmt = "%H:%M"
        start_dt = datetime.strptime(start_time_str, fmt)
        end_dt = start_dt + timedelta(minutes=duration_minutes)
        return end_dt.strftime(fmt)

```

#endregion

```
#region search
class ExistingJobsDispatcher(MessageHandler):
    def handle(self, msg:Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()

                cur.execute("SELECT DISTINCT profession FROM professional")
                rows = cur.fetchall()
                if not rows:
                    return MessageTypes.GET_JOBS, {StatusMessage.FAILED_TO_GET_JOBS.value:"Our professionals are still not verified in the system, sorry!"}

                jobs = []
                for row in rows:
                    jobs.append(row[0])

                jobs.sort()
                return MessageTypes.GET_JOBS, {StatusMessage.GOT_JOBS.value:{"jobs":jobs}}

        except Exception as e:
            print("Existing job dispatcher error - ",e)
            return MessageTypes.GET_JOBS, {StatusMessage.FAILED_TO_GET_JOBS.value:"Failed retrieving info, reload page"}

class CitiesByJobDispatcher(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()

                profession = msg.data["job"]

                cur.execute("SELECT DISTINCT service_cities FROM professional WHERE profession =?", (profession,))
                rows = cur.fetchall()
                if not rows:
                    return MessageTypes.GET_CITIES_BY_JOB, {StatusMessage.FAILED_TO_GET_CITIES.value:"Server error"}

                unique_cities = set()

                for row in rows:
                    city_string = row[0]
                    if city_string:
                        parts = [c.strip() for c in city_string.split(',')]
                        unique_cities.update(parts)

                cities = sorted(list(unique_cities))

                return MessageTypes.GET_CITIES_BY_JOB, {StatusMessage.GOT_CITIES.value:{"cities":cities}}

        except Exception as e:
            print("Cities find by job dispatcher exception - ", e)
            return MessageTypes.GET_CITIES_BY_JOB, {StatusMessage.FAILED_TO_GET_CITIES.value:"Couldn't retrieve cities, refresh page"}

class MinimalProfessionalInfo(MessageHandler):
```



```

def handle(self, msg:Message) -> tuple:
    try:
        with sqlite3.connect(DATABASE) as conn:
            cur = conn.cursor()

            city = msg.data["city"]
            job = msg.data["job"]

            search_term = f"%{city}%"

            cur.execute("""SELECT id, description, rating, hour_price FROM professional
                           WHERE profession=? AND (' ' || REPLACE(service_cities, ' ', ' ', ' ')
                           || ' ') LIKE ? """, (job, search_term))

            rows = cur.fetchall()
            if not rows:
                return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {StatusMessage.FAILED_TO_
GET_MINIMAL_INFO.value:"Error fetching info"}

            users = defaultdict(dict)
            for row in rows:
                id, description, rating, hour_price = row
                users[id] = {"description":description, "rating":rating, "price":hour_pric
e}

            if users:
                dict(users)
                return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {StatusMessage.GOT_MINIMAL_IN
FO.value:users}

    except Exception as e:
        print("Minimal professional info error - ",e)
        return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {StatusMessage.FAILED_TO_GET_MINI
MAL_INFO.value:"Error fetching info"}

class FreelancerPublicInfo(MessageHandler):
    def handle(self, msg:Message) ->tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()

                cur.execute("""
                    SELECT u.id, p.profession, p.service_cities, p.description, p.years_experie
nce, p.avg_job_duration,
                    p.rating, p.hour_price, u.full_name FROM professional p INNER JOIN users u
ON p.user_id = u.id
                    WHERE p.id = ?
                    """, (msg.data["id"],))

                row = cur.fetchone()
                if not row:
                    return MessageTypes.GET_PUBLIC_PROFILE_INFO, {StatusMessage.FAILED_TO_GET_
FREELANCER_PUBLIC_INFO.value:"Couldn't get freelancer's info"}

                id, profession, cities, description, years, job_duration, rating, price, name
= row

                to_send = {"id":id, "job":profession, "cities":cities,
                           "description":description, "years":years,
                           "job_duration":job_duration, "rating":rating,
                           "price":price, "username":name}

```

```

        return MessageType.GET_PUBLIC_PROFILE_INFO, {StatusMessage.GOT_FREELANCER_PUBLIC_INFO.value:to_send}

    except Exception as e:
        print("Freelancer public info exception - ", e)
        return MessageType.GET_PUBLIC_PROFILE_INFO, {StatusMessage.FAILED_TO_GET_FREELANCER_PUBLIC_INFO.value:"Couldn't get freelancer's info"}

#endregion

#endregion

#region Signup Validators
class Validator(ABC):
    @abstractmethod
    def validate(self, msg:Message) ->tuple[bool,str]:
        pass

class UserSignUpValidator(Validator):
    def validate(self, msg:Message) ->tuple[bool, str]:
        #Password I don't check
        #Name I don't check
        try:
            email = msg.data["email"]
            full_name = msg.data["name"]
            if not self._check_email_format(email):
                return False, {StatusMessage.FAILED_SIGN_UP.value:"Email format is incorrect"}
            if len(full_name)>30:
                return False, {StatusMessage.FAILED_SIGN_UP.value:"Name field can't exceed 30
characters"}
            return True, ""
        except Exception as e:
            print("User sign up validator - ",e)
            return False, {StatusMessage.FAILED_SIGN_UP.value:"Not all fields were sent"}

    def _check_email_format(self, email):
        pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
        return re.match(pattern, email) is not None

class FreelancerSignUpValidator(Validator):
    def validate(self, msg:Message) ->tuple[bool, str]:
        #Password I don't check
        #Name I don't check
        status = False
        error_message = StatusMessage.FAILED_SIGN_UP.value

        data = msg.data
        try:
            email = data["email"]
            if not self._check_email_format(email):
                return status, {error_message:"Email format is incorrect"}

            if len(data["name"]) > 30:

```

```

        return status, {error_message:"Name field can't exceed 30 characters"}

    if data["profession"] not in PROFESSIONS:
        return status, {error_message:"Job isn't recognized"}

    if not self._check_city(data["cities"]):
        return status, {error_message:"City not recognized"}

    if data["years"] > 40:
        return status, {error_message:"Experience seems a bit off..."}

    try:
        start_dt = datetime.strptime(data["startWorking"], "%H:%M")
        finish_dt = datetime.strptime(data["finishWorking"], "%H:%M")
        job_duration_dt = datetime.strptime(data["jobDuration"], "%H:%M")
        job_duration = timedelta(hours=job_duration_dt.hour, minutes=job_duration_dt.m
inute)
    except:
        return status, {error_message:"Work, start, or finish time bad format"}

    if start_dt >= finish_dt:
        return status, {error_message:"Start work time must be before finish work time"}
}

    if job_duration.total_seconds() <= 0 or job_duration >= timedelta(days=1):
        return status, {error_message:"Enter proper job duration"}

    first_job_time = start_dt + job_duration
    if first_job_time > finish_dt:
        return status, {error_message:"You won't complete a single job..."}

    if data["role"] not in ["Freelancer", "User"]:
        return status, {error_message:"No such user type"}

    if type(data["hourPrice"]) != int:
        return status,{error_message:"price per hour is not in correct format "}

    if data["hourPrice"] > 1000:
        return status, {error_message: "your wage is to expensive for this site"}

    if len(data["description"]) > 500:
        return status, {error_message:f"Description must be less then 500 characters,
yours is {len(data['description'])}"}

    status = True
    return status, {StatusMessage.SIGNING_UP.value:""}
except Exception as e:
    print("Freelancer validator - ", e)
    return False, {StatusMessage.FAILED_SIGN_UP.value:"Not all fields were sent"}

def _check_city(self, cities):
    if type(cities) != list:
        return False

```

```

        for city in cities:
            if city not in LOCALITIES:
                return False
        return True

def _check_email_format(self, email):
    pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
    return re.match(pattern, email) is not None
#endregion

class EmailVerification:
    def __init__(self, email):
        self.email = email
        self.verification_code = f'{random.randint(0, 999999):06d}'
        self.expires_at = datetime.now()+timedelta(minutes=5)

    def send_verification_code(self):
        message = MIMEMultipart("alternative")

        message["Subject"] = "Verification email"
        message["From"] = EMAIL
        message["To"] = self.email

        html = f""" <html>
            <body style="font-family: Arial, sans-serif; color: #333;">
                <div style="max-width: 600px; margin: auto; padding: 20px; border:
1px solid #ddd; border-radius: 10px; background-color: #f9f9f9;">
                    <h2 style="color: #2a9d8f;">Welcome to Pro-Find!</h2>
                    <p>Hi there,</p>
                    <p>Thanks for signing up. Your verification code is:</p>
                    <p style="font-size: 24px; font-weight: bold; color: #e76f51;">{se
lf.verification_code}</p>
                    <p>Please enter this code in the app to verify your account.</p>
                    <hr>
                    <p style="font-size: 12px; color: #888;">If you didn't sign up, yo
u can ignore this email.</p>
                </div>
            </body>
        </html>
        """

        text = f"Hi there!\nYour verification code is {self.verification_code}\nPlease enter it
in the app."
        message.attach(MIMEText(text, "plain"))
        message.attach(MIMEText(html, "html"))

    try:
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
            server.login(EMAIL, AUTHENTICATION_PAS)
            server.sendmail(EMAIL, self.email, message.as_string())
            print("Email sent successfully!")
            return True
    except Exception as e:
        print(f"Error: {e}")
        return False

```

```

@staticmethod
def check_verification_code(msg:Message) -> tuple [bool, dict]:
    try:
        email = msg.data["email"]
        veri_code = msg.data["verification_code"]
        with sqlite3.connect(DATABASE) as conn:
            cur = conn.cursor()
            cur.execute("SELECT verification_code, expires_at FROM pending_users WHERE email=?", (email,))
            data = cur.fetchone()
            if not data:
                return False, {StatusMessage.VERIFICATION_BAD.value:"Email not found"}

            else:
                expires_at = datetime.strptime(data[1], "%Y-%m-%d %H:%M:%S.%f")
                veri_code_fetched = data[0]

                if expires_at < datetime.now():
                    cur.execute("DELETE FROM pending_users WHERE email=?", (email,))

                    conn.commit()
                    return False, {StatusMessage.VERIFICATION_BAD.value:"Verification time
expired, restart the process"}

                if veri_code_fetched == veri_code:
                    return True, {StatusMessage.VERIFICATION_GOOD.value:""}

                else:
                    return False, {StatusMessage.VERIFICATION_BAD.value:"Verification code
does not match"}

    except Exception as e:
        print (f"Verification code check - {e}")
        return False, {StatusMessage.VERIFICATION_BAD.value:"Not all fields were sent"}

```

```

import socket
import hashlib
import base64
from enum import Enum
import json
from message_types import MessageTypes
import ssl

#region ---> Handles the WebSocket handshake

def recv_http_handshake_msg(soc:socket.socket) ->bytes:
    data=b""
    while b'\r\n\r\n' not in data:
        chunk=soc.recv(1024)
        if not chunk:
            return data
        data+=chunk
    return data

class HttpMessage:
    def __init__(self, http_method, protocol_version, host, upgrade,connection, websocket_key):
        self.http_method = http_method
        self.protocol_version = protocol_version
        self.host = host
        self.upgrade = upgrade
        self.connection = connection
        self.websocket_key = websocket_key

    def __repr__(self):
        return(
            f"Http method      : {self.http_method}\n"
            f"Protocol version    : {self.protocol_version}\n"
            f"Server              : {self.host}\n"
            f"Upgrade to          : {self.upgrade}\n"
            f"Connection type      : {self.connection}\n"
            f"WebSocket key       : {self.websocket_key}"
        )

class HttpMessageParser:

    @staticmethod
    def parse_http_message(msg: bytes) -> HttpMessage:
        headers = msg.split(b'\r\n\r\n')[0].decode()
        lines = headers.split('\r\n')
        http_method, _, protocol_version = lines[0].split(' ')

        header_dict = {}
        for line in lines[1:]:
            key, value = line.split(": ", 1)
            header_dict[key] = value
        try:
            return HttpMessage(
                http_method,
                protocol_version,
                host=header_dict["Host"],
                upgrade=header_dict["Upgrade"],
                connection=header_dict["Connection"],
                websocket_key=header_dict["Sec-WebSocket-Key"],
            )
        except:
            return None

```

```

    )
    except Exception as e:
        print(f"Found exception ---- {e}")
        return None

#Build the Http Upgrade message
class HttpUpgrade:

    def __init__(self, http_message:HttpMessage):
        self.message = http_message

    def to_upgrade_to_WebSocket(self) -> bool:
        if self.message.connection != "Upgrade":
            return False
        elif self.message.upgrade != "websocket":
            return False
        else:
            return True

    def upgrade_response(self) -> str:
        GUID = "258EAF5-E914-47DA-95CA-C5AB0DC85B11"
        contecation:str = self.message.websocket_key.strip()+GUID
        sha1_hash = hashlib.sha1(contecation.encode()).digest()
        accept_text = base64.b64encode(sha1_hash).decode("ASCII")

        version = "HTTP/1.1 "
        accept_code = "101 Switching Protocols\r\n"
        upgrade = f"Upgrade: {self.message.upgrade}\r\n"
        connection = f"Connection: {self.message.connection}\r\n"
        accept_key = f"Sec-WebSocket-Accept: {accept_text}\r\n"
        return_text = f"{version}{accept_code}{upgrade}{connection}{accept_key}\r\n"
        return return_text

#endregion

class WebSocketOpcodes(Enum):
    CONTINUATION = 0x0
    TEXT = 0x1
    BINARY = 0x2
    CLOSE_CONNECTION = 0x8
    PING_MESSAGE = 0x9
    PONG_MESSAGE = 0xA

#Builds WebSocket messages
class WebSocketMessageFactory:
    def __init__(self, opcode:WebSocketOpcodes, to_split=None, payload=None): #All types of me
ssage except for pong
        self.opcode:WebSocketOpcodes = opcode
        self.to_split = to_split
        self.payload = payload
        if self.payload ==None:
            self.payload = ''

        if self.opcode != WebSocketOpcodes.PONG_MESSAGE and to_split==None:
            raise RuntimeError("Expected to recieve 'to_split' value but none was given")
        else:
            self.message_to_send = self._build_pong_message() if self.opcode == WebSocketOpcodes.PONG_MESSAGE else self._build_text_message()

```

```

ES.PONG_MESSAGE else self._build_websocket_message()

def __repr__(self):
    to_print = f"Message content type - {self.opcode}"
    if self.to_split != None:
        to_print+=f"\nMessage needs to be splited - {'True' if self.to_split else 'False'}"
    "
    if self.payload:
        to_print+=f"\nMessage content - {self.payload}"

    to_print+=f"\nMessage to be sent - {self.message_to_send}"
    return to_print

def _build_websocket_message(self) ->bytes:
    frame_bytes = bytearray()

    fin = 0 if self.to_split else 1
    frame_bytes.append((fin<<7)|self.opcode.value)

    mask = 0
    bytes_payload = self.payload if type(self.payload)==bytes else self.payload.encode("UTF
F-8")

    def get_length_bits(bytes_payload):
        payload_length = len(bytes_payload)

        if payload_length<=125:
            length_field = payload_length
            extended_length_bytes = b''

        elif payload_length<=65535:
            length_field = 126
            extended_length_bytes = payload_length.to_bytes(2, "big")

        else:
            length_field = 127
            extended_length_bytes = payload_length.to_bytes(8, "big")

        return length_field, extended_length_bytes

    length_field, extend_length_bytes = get_length_bits(bytes_payload)
    frame_bytes.append((mask<<7)|length_field)
    frame_bytes.extend(extend_length_bytes)
    frame_bytes.extend(bytes_payload)
    return (bytes(frame_bytes))

def _build_pong_message(self):
    frame_bytes = bytearray()
    fin = 1
    frame_bytes.append((fin<<7)|WebSocketOpcodes.PONG_MESSAGE.value)

    mask = 0
    if self.payload:
        bytes_payload = self.payload if type(self.payload)==bytes else self.payload.encode
("UTF-8")
        payload_length = len(bytes_payload)
        frame_bytes.append((mask<<7)|payload_length)
        frame_bytes.extend(bytes_payload)

```



```

else:
    payload_length = 0
    frame_bytes.append((mask<<7)|payload_length)

return(bytes(frame_bytes))

```

#region ---> WebSocket message parser

```

class WebSocketFrame:
    def __init__(self, fin, opcode, masked, payload_length, payload, masking_key=None):
        self.message_is_split = False if fin == 1 else True
        self.opcode = WebSocketOpcodes(opcode)
        self.masked = bool(masked)
        self.payload_length = payload_length
        self.payload = payload
        if masking_key:
            self.masking_key = masking_key

    def __repr__(self):
        to_print = ""
        if self.message_is_split:
            to_print+="Message is splitted"
        else:
            to_print+="This is the whole message"
        to_print+=f"\nMessage type :{self.opcode}"
        to_print+=f"\nthis message is {'masked' if self.masked else 'not masked'}"
        to_print+=f"\nThe payload length is :{self.payload_length}"
        to_print+=f"\nPayload :{self.payload}"
        if self.masking_key:
            to_print+=f"\nMasking key :{self.masking_key}"
        return to_print

```

```

class WebSocketMessageParser:
    @staticmethod
    def _parse_one_frame(clt:socket.socket) ->WebSocketFrame:
        first_byte = clt.recv(1)[0]
        fin = (first_byte>>7) & 1
        opcode = first_byte & 0x0F

        second_byte = clt.recv(1)[0]
        masked = (second_byte>>7) & 1

        payload_len = second_byte & 0x7F
        if payload_len == 126:
            bytes_of_extended = clt.recv(2)
            payload_len = int.from_bytes(bytes_of_extended, "big")
        elif payload_len == 127:
            bytes_of_extended = clt.recv(8)
            payload_len = int.from_bytes(bytes_of_extended, "big")

        if masked:
            mask = clt.recv(4)
        else:
            mask= None

        payload = b""
        bytes_to_read = payload_len

```

```

        while bytes_to_read>0:
            chunk = clt.recv(bytes_to_read)
            if not chunk:
                raise RuntimeError("Connection closed by the client")
            payload+=chunk
            bytes_to_read-=len(chunk)

        if masked:
            payload = bytes(b ^ mask[i % 4] for i, b in enumerate(payload))

        return WebSocketFrame(fin, opcode, masked, payload_len, payload, masking_key=mask)

    @staticmethod
    def parse_message(clt:socket.socket) ->bytes:
        buffer = b''

        while True:
            frame = WebSocketMessageParser._parse_one_frame(clt)

            if frame.opcode in {WebSocketOpcodes.CLOSE_CONNECTION,
                               WebSocketOpcodes.PING_MESSAGE,
                               WebSocketOpcodes.PONG_MESSAGE}:
                if frame.payload_length>=125 or frame.message_is_split:
                    return b'FAULTY FRAME'
                if frame.opcode ==WebSocketOpcodes.CLOSE_CONNECTION:
                    clt.close()
                    return b"CLIENT CLOSED"
                elif frame.opcode==WebSocketOpcodes.PING_MESSAGE:
                    to_send = WebSocketMessageFactory(WebSocketOpcodes.PONG_MESSAGE, payload=f
rame.payload)
                    clt.send(to_send.message_to_send)
                    continue

            if frame.opcode not in {WebSocketOpcodes.CONTINUATION,
                                    WebSocketOpcodes.TEXT,
                                    WebSocketOpcodes.BINARY} or (frame.opcode == WebSocketOpcodes.CONTINUATION and buffer == b''):
                return b"FAULTY FRAME"

            buffer+=frame.payload

            if not frame.message_is_split:
                break

        return buffer

#endregion

#region accept client
def accept_client(clt_soc):
    new_soc = accept_TLS_encryption(clt_soc)
    accept_websocket_upgrade_request_from_client(new_soc)
    return new_soc

def accept_TLS_encryption(clt_soc):
    CERTIFICATE_PATH = r"C:\Coding\pro-find\certificate\server.crt"
    KEY_PATH = r"C:\Coding\pro-find\certificate\server.key"
    # CERTIFICATE_PATH =r"D:\pro-find\certificate\server.crt"
    # KEY_PATH = r"D:\pro-find\certificate\server.key"
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)

```

```

context.verify_mode = ssl.CERT_NONE
context.load_cert_chain(certfile=CERTIFICATE_PATH, keyfile=KEY_PATH)
tls_soc = context.wrap_socket(clt_soc, server_side=True)
return tls_soc

def accept_websocket_upgrade_request_from_client(clt_soc: socket.socket):
    info = recv_http_handshake_msg(clt_soc)
    http_message = HttpMessageParser.parse_http_message(info)

    accept_WebSocket_upgrade = HttpUpgrade(http_message)
    if accept_WebSocket_upgrade.to_upgrade_to_WebSocket():
        to_send = accept_WebSocket_upgrade.upgrade_response()
        clt_soc.send(to_send.encode())
        print("Connected")
#endregion

def build_proper_json_payload(type: MessageTypes, payload, token):
    type_of_message = type.value
    type_of_data = "JSON"

    if isinstance(payload, bytes):
        safe_payload = base64.b64encode(payload).decode("utf-8")
        type_of_data = "BYTES"
    else:
        try:
            json.dumps(payload)
            safe_payload = payload
        except (TypeError, OverflowError):
            safe_payload = str(payload)

    return {
        "type": type_of_message,
        "data": safe_payload,
        "data_type": type_of_data,
        "token": token
    }

def send_message(clt: socket.socket, type, token, msg=None, to_split_message=None, opcode: WebSocketOpcodes=WebSocketOpcodes.TEXT):
    try:
        msg_dict = build_proper_json_payload(type, msg, token) if msg is not None else None

        msg_to_send = json.dumps(msg_dict)

        message = WebSocketMessageFactory(opcode, to_split=to_split_message, payload=msg_to_send)

        clt.send(message.message_to_send)
        return True

    except Exception as e:
        print("Failed to send message:", e)
        return False

```

```
def recv_message(clt: socket.socket):
    try:
        parsed_message = WebSocketMessageParser.parse_message(clt)
        if parsed_message == b'FAULTY FRAME' or parsed_message == b"CLIENT CLOSED":
            return None
        else:
            return json.loads(parsed_message)
    except Exception as e:
        print(e)
        return None
```